

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To understand the process of convolution over the image and apply over the classification problem	
Experiment No: 4	Date:	Enrolment No: 92410133039

ii

Aim: To understand the process of convolution over the image and apply it to a classification problem using CNN.

IDE: Google Collab

Theory:

Convolutional Neural Networks (CNN) are a type of deep learning model mainly used for image classification and recognition. They work by extracting important features from images using layers like convolution, pooling, and fully connected layers. CNNs provide high accuracy because they automatically learn features such as edges, shapes, and patterns from images.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Normalize the data
5. Pre-process the data
6. Visualize the Data
7. Write the CNN model function
8. Write the Cost Function
9. Write the Gradient Descent optimization algorithm
10. Apply the training over the dataset to minimize the loss
11. Observe the cost function vs iterations learning curve

Program (Code):

```
import tensorflow as tf

from tensorflow.keras import layers, models

from tensorflow.keras.datasets import cifar10

from tensorflow.keras.utils import to_categorical

import matplotlib.pyplot as plt
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To understand the process of convolution over the image and apply over the classification problem	
Experiment No: 4	Date:	Enrolment No: 92410133039

ii

```
(train_images, train_labels), (test_images, test_labels) = cifar10.load_data()
```

```
train_images, test_images = train_images / 255.0, test_images / 255.0
```

```
train_labels = to_categorical(train_labels, 10)
```

```
test_labels = to_categorical(test_labels, 10)
```

```
model = models.Sequential([
```

```
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
```

```
    layers.MaxPooling2D(pool_size=(2, 2)),
```

```
    layers.Flatten(),
```

```
    layers.Dense(128, activation='relu'),
```

```
    layers.Dense(10, activation='softmax')
```

```
])
```

```
model.compile(optimizer='adam',
```

```
    loss='categorical_crossentropy',
```

```
    metrics=['accuracy'])
```

```
history = model.fit(train_images, train_labels, epochs=10, batch_size=64, validation_data=(test_images, test_labels))
```

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To understand the process of convolution over the image and apply over the classification problem	
Experiment No: 4	Date:	Enrolment No: 92410133039

ii

```
print(f"Test accuracy: {test_acc * 100:.2f}%")
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
```

```
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
```

```
plt.title('Model Accuracy')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Accuracy')
```

```
plt.legend(loc='lower right')
```

```
plt.show()
```

```
plt.plot(history.history['loss'], label='Training Loss')
```

```
plt.plot(history.history['val_loss'], label='Validation Loss')
```

```
plt.title('Model Loss')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Loss')
```

```
plt.legend(loc='upper right')
```

```
plt.show()
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To understand the process of convolution over the image and apply over the classification problem	
Experiment No: 4	Date:	Enrolment No: 92410133039

ii

Results:

```
[1] import tensorflow as tf
    from tensorflow.keras import layers, models
    from tensorflow.keras.datasets import cifar10
    from tensorflow.keras.utils import to_categorical
    import matplotlib.pyplot as plt

[2] (train_images, train_labels), (test_images, test_labels) = cifar10.load_data()
    train_images, test_images = train_images / 255.0, test_images / 255.0
    train_labels = to_categorical(train_labels, 10)
    test_labels = to_categorical(test_labels, 10)

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 — 119s 1us/step

[3] model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

[4] model.compile(optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy'])
```

```
metrics=['accuracy'])

[5] history = model.fit(train_images, train_labels, epochs=10, batch_size=64, validation_data=(test_images, test_labels))

Epoch 1/10
782/782 — 49s 59ms/step - accuracy: 0.4823 - loss: 1.4498 - val_accuracy: 0.5467 - val_loss: 1.2786
Epoch 2/10
782/782 — 40s 51ms/step - accuracy: 0.5926 - loss: 1.1693 - val_accuracy: 0.5887 - val_loss: 1.1626
Epoch 3/10
782/782 — 39s 50ms/step - accuracy: 0.6338 - loss: 1.0492 - val_accuracy: 0.6093 - val_loss: 1.1120
Epoch 4/10
782/782 — 39s 50ms/step - accuracy: 0.6681 - loss: 0.9533 - val_accuracy: 0.6344 - val_loss: 1.0339
Epoch 5/10
782/782 — 39s 50ms/step - accuracy: 0.6911 - loss: 0.8797 - val_accuracy: 0.6419 - val_loss: 1.0252
Epoch 6/10
782/782 — 39s 50ms/step - accuracy: 0.7165 - loss: 0.8161 - val_accuracy: 0.6419 - val_loss: 1.0472
Epoch 7/10
782/782 — 38s 49ms/step - accuracy: 0.7361 - loss: 0.7576 - val_accuracy: 0.6513 - val_loss: 1.0047
Epoch 8/10
782/782 — 39s 50ms/step - accuracy: 0.7556 - loss: 0.7036 - val_accuracy: 0.6537 - val_loss: 1.0251
Epoch 9/10
782/782 — 41s 52ms/step - accuracy: 0.7730 - loss: 0.6546 - val_accuracy: 0.6491 - val_loss: 1.0738
Epoch 10/10
782/782 — 38s 49ms/step - accuracy: 0.7890 - loss: 0.6057 - val_accuracy: 0.6478 - val_loss: 1.0609

[6] test_loss, test_acc = model.evaluate(test_images, test_labels)
    print(f"Test accuracy: {test_acc * 100:.2f}%")

313/313 — 3s 10ms/step - accuracy: 0.6478 - loss: 1.0609
Test accuracy: 64.78%
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To understand the process of convolution over the image and apply over the classification problem	
Experiment No: 4	Date:	Enrolment No: 92410133039

ii

```

782/782 ————— 38s 49ms/step - accuracy: 0.7890 - loss: 0.6057 - val_accuracy: 0.6478 - val_loss: 1.0609

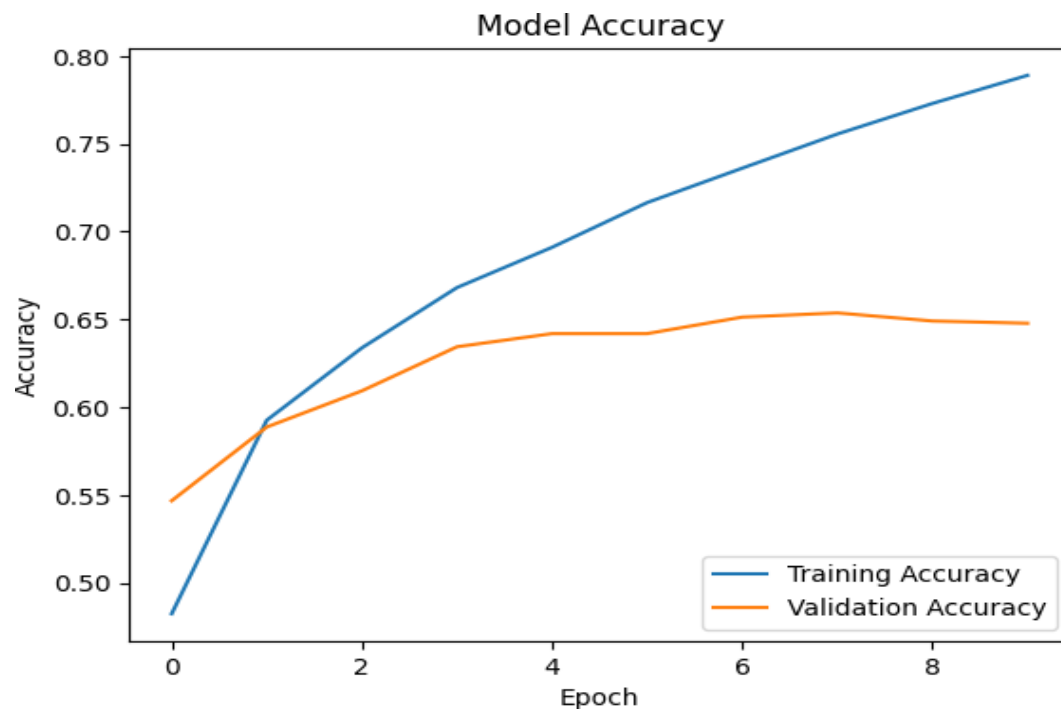
[6]
✓ 3s
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"Test accuracy: {test_acc * 100:.2f}%")

313/313 ————— 3s 10ms/step - accuracy: 0.6478 - loss: 1.0609
Test accuracy: 64.78%

[7]
✓ 0s
▶ plt.plot(history.history['accuracy'], label='Training Accuracy')
  plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
  plt.title('Model Accuracy')
  plt.xlabel('Epoch')
  plt.ylabel('Accuracy')
  plt.legend(loc='lower right')
  plt.show()

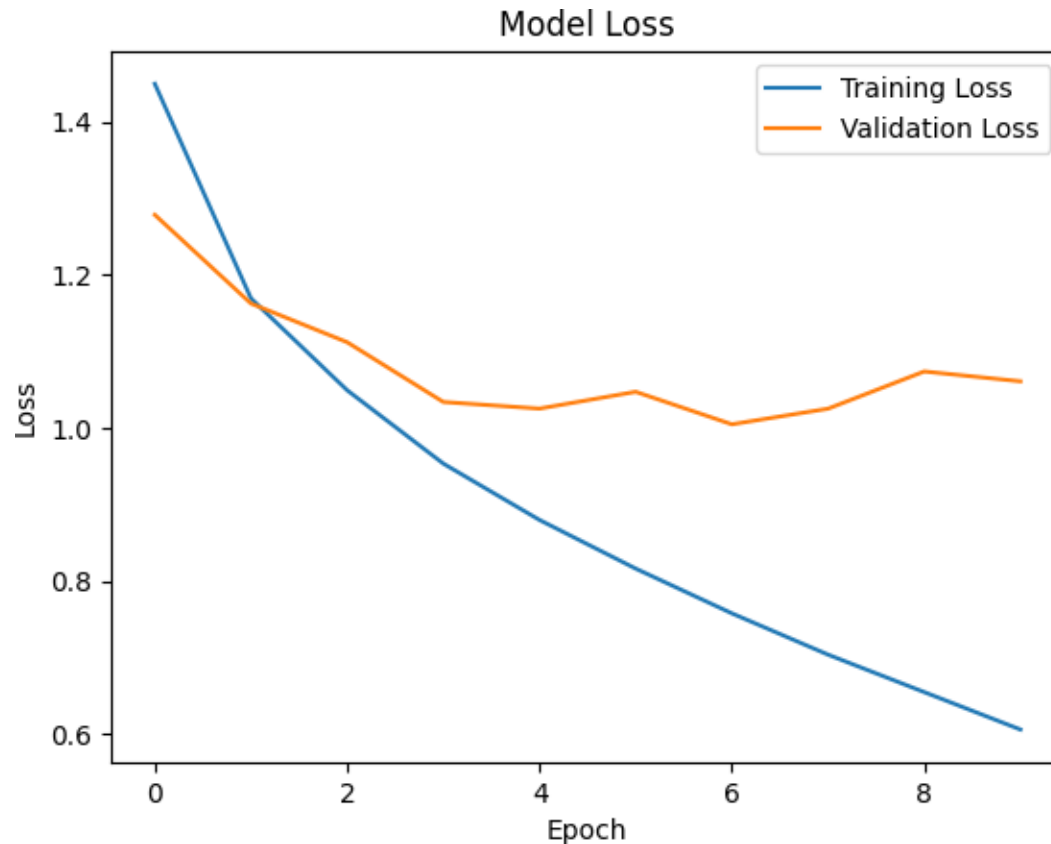
  plt.plot(history.history['loss'], label='Training Loss')
  plt.plot(history.history['val_loss'], label='Validation Loss')
  plt.title('Model Loss')
  plt.xlabel('Epoch')
  plt.ylabel('Loss')
  plt.legend(loc='upper right')
  plt.show()

```



 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To understand the process of convolution over the image and apply over the classification problem	
Experiment No: 4	Date:	Enrolment No: 92410133039

ii



Pre Lab Exercise:

a. **What is Convolution process? Explain giving example.**

Convolution is a process in which a small matrix called a filter or kernel is applied over an input image to extract important features. The filter slides over the image and performs element-wise multiplication followed by summation.

Example:

If we apply an edge-detection filter on an image, the convolution operation will highlight edges by detecting intensity changes in pixel values.

b. **What are different layers in CNN model?**

CNN consists of three main layers:

1. **Convolutional Layer** – Extracts features from the image using filters.
2. **Pooling Layer** – Reduces the size of the feature map to decrease computation.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To understand the process of convolution over the image and apply over the classification problem	
Experiment No: 4	Date:	Enrolment No: 92410133039

ii

3. **Fully Connected Layer** – Takes extracted features and predicts the final output (classification).

c. **What is the requirement of the pooling layer in the CNN model?**

Pooling layer is used to reduce the spatial size of feature maps. This helps in:

- Reducing computational cost
- Avoiding overfitting
- Extracting dominant features

Max pooling also removes noise by selecting the maximum value.

d. **What is the requirement of the use of ReLU activation function after convolution step?**

ReLU is used to remove negative values from the feature map.

It converts all negative values to zero and keeps positive values unchanged.

This helps in:

- Introducing non-linearity
- Speeding up training
- Improving model performance

Observation and Result Analysis:

e. **Nature of the dataset**

The dataset consists of image data used for classification tasks. It contains labeled images which are preprocessed and normalized before training. The dataset is divided into training and validation sets

f. **Training Process without regularization**

Without regularization:

- Model learns training data very well
- Training accuracy increases rapidly
- Validation accuracy may be low
- Overfitting occurs (model memorizes data instead of learning)

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To understand the process of convolution over the image and apply over the classification problem	
Experiment No: 4	Date:	Enrolment No: 92410133039

ii

g. After regularization in the training Process

After applying regularization:

- Overfitting is reduced
- Training accuracy slightly decreases
- Validation accuracy improves
- Model becomes more generalized

h. Observation over the Learning Curves

- Training accuracy increases with epochs
- Validation accuracy stabilizes after some time
- Loss decreases gradually
- Without regularization → gap between training & validation
- With regularization → curves become closer (better performance)

Post Lab Exercise:

a. Why CNN is preferred over ANN for images

CNN is preferred because:

It automatically extracts features (no manual feature extraction needed)

- It handles spatial data efficiently
- It requires fewer parameters than ANN
- Provides higher accuracy for image tasks

b. Can CNN be applied over Text data? If yes, then how. If no, then why?

Yes, CNN can be applied to text data.

How:

- Text is converted into numerical form (word embeddings)

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To understand the process of convolution over the image and apply over the classification problem	
Experiment No: 4	Date:	Enrolment No: 92410133039

ii

- Convolution is applied to detect patterns like phrases
- Used in tasks like sentiment analysis and text classification

c. What is the role of dropout layer?

Dropout layer randomly disables some neurons during training.

Purpose:

- Prevents overfitting
- Improves generalization
- Makes the model more robust

d. What will happen if maxpooling is replaced with minpooling?

If max pooling is replaced with min pooling:

- Important features may be lost
- Model performance decreases
- Noise may increase
- Accuracy will be lower

Because max pooling selects the most important features, while min pooling selects least important values.

Conclusion:-

The experiment successfully demonstrated the working of Convolutional Neural Networks for image classification. We understood the role of convolution, pooling, and fully connected layers in extracting and classifying features. The use of ReLU activation improved model performance, and regularization techniques helped reduce overfitting. Overall, CNN proved to be an efficient and accurate model for image-based classification tasks.